ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Artificial Intelligence with Python

Prof. Dr.-Ing. Wei Yap Tan

12–20 November 2026

About Me

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

- **Name:** Prof. Dr.-Ing. Wei Yap Tan
- **Home Institution:** Technical University of Applied Sciences Mannheim, Germany
- **Fields of Research and Lectures:** Sensor technology, artificial intelligence, electronics design
- **E-Mail:** w.tan@th-mannheim.de



Technische Hochschule
mannheim

General Information

General Information

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

- **Format:** 1 h lecture + 3 h lab per teaching unit
- **Workload:** 7 h lectures + 21 h labs and projects
- **Prerequisites:**
  - No prior Python, AI, or ML knowledge required
  - Programming experience assumed (e.g. Java)
- **Progression:** Python → ML workflow → models & evaluation
- **Final part:** self-defined ML project and presentation

Course Content

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

1. Introduction to Artificial Intelligence and Machine Learning
2. Python for Data Science
3. The Machine Learning Workflow
4. How Machine Learning Models Learn
5. Classical ML Methods and Evaluation
6. Neural Networks and Deep Learning
7. Machine Learning Project and Presentation

Introduction to AI and ML

What is Artificial Intelligence?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Artificial Intelligence

Artificial Intelligence (AI) is the field of creating computer systems capable of performing tasks that normally require human intelligence.

Examples include:

- recognizing objects in images
- understanding and generating language
- making predictions and decisions
- solving complex problems
- learning from experience or data

What Do We Mean by “Intelligence”?

AI systems may exhibit different aspects of intelligent behavior:

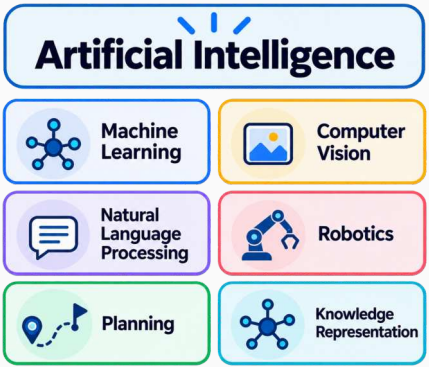
- **Perception**
  Images, sound, sensor data
- **Reasoning**
  Drawing conclusions from information
- **Learning**
  Improving from data or experience
- **Language**
  Understanding and generating text
- **Decision making**
  Choosing actions based on a goal
- **Problem solving**
  Finding solutions to complex tasks

AI Is a Broad Field

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Artificial Intelligence includes many different approaches and applications.

- Machine Learning
- Computer Vision
- Natural Language Processing
- Robotics
- Planning and Search
- Knowledge Representation



AI Does Not Necessarily Mean "Human-Like"

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

- Most AI systems are designed for a **specific task**.
- They can perform very well without understanding the task in the same way a human does.
- An AI system may exceed human performance in one task while being unable to perform very simple tasks outside its domain.

Example

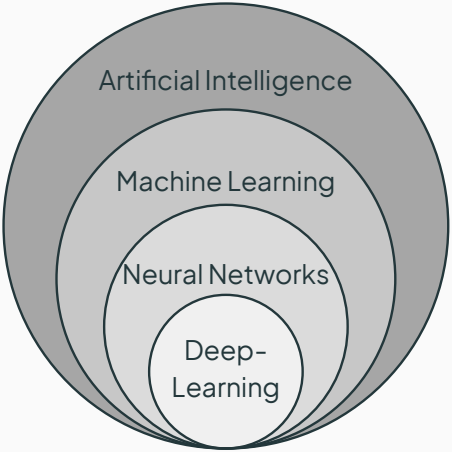
A chess-playing AI may defeat world-class players, but it cannot recognize a cat, translate a sentence, or drive a car unless it was specifically designed or trained for those tasks.

AI is about intelligent behavior in machines, not necessarily human intelligence.

AI, Machine Learning, and Deep Learning

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

- Machine learning is a subset of artificial intelligence.
- ML learns from examples and tries to extract the underlying patterns.
- ML should however **not** learn the examples by heart.



Artificial Intelligence

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Artificial Intelligence

AI is the broad goal of building systems that can perform tasks associated with intelligent behavior.

AI can be implemented using many different approaches:

- rule-based systems
- search and planning
- knowledge representation
- machine learning
- combinations of multiple techniques

Key Point

Not every AI system learns from data.

Machine Learning

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Machine Learning

Machine Learning (ML) is an approach to AI in which systems **learn patterns from data** rather than relying only on explicitly programmed rules.

Instead of writing all decision rules manually:

Data + Learning Algorithm
⇓
Model

The model can then be used to make predictions or decisions on new data.

Deep Learning

Deep Learning is a type of Machine Learning based on **multi-layer artificial neural networks**.

Deep learning is especially successful for:

- image recognition
- speech processing
- natural language processing
- generative AI

How Are They Related?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

AI	Broad field concerned with intelligent computer systems.
Machine Learning	AI approach where systems learn from data.
Deep Learning	ML approach using multi-layer neural networks.

Important

Machine Learning is not the same as AI, and Deep Learning is not the same as Machine Learning. They describe different levels within the same broader field.

Traditional Programming vs. Machine Learning

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Transitionnal Programming

Data + Rules
⇓
Output

- Rules are written explicitly
- Behavior is determined by the program
- Well suited for clearly defined logic

Machine Learning

Data + Expected Output
⇓
Learned Model

- Rules are learned from examples
- Behavior emerges from training
- Useful when rules are difficult to specify manually

A Simple Example: Spam Mail Detection

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Task

Decide whether an email is spam.

Traditional approach

- Define suspicious keywords
- Count links
- Check sender information
- Combine rules manually

```
if links > 5 and ...
```

Machine learning approach

- Collect labeled emails
- Extract useful features
- Train a model
- Let the model learn predictive patterns

Where Does the “Programming” Move?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Machine Learning does **not** mean that we stop programming.

We still need to program:

- data collection and preprocessing
- feature preparation
- model selection and training
- evaluation
- deployment and integration

The difference is that part of the system’s behavior is no longer encoded directly as hand-written rules.

When Is Machine Learning Useful?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Machine learning is particularly useful when:

- the decision rules are too complex to write explicitly
- large amounts of example data are available
- useful patterns exist in the data
- the task may change over time

Examples:

- image recognition
- recommendation systems
- predictive maintenance

But...

If a task can be solved reliably with a few simple rules, machine learning may be unnecessary.

From a Java Programmer's Perspective

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A familiar Java program might look like:

$$\texttt{input} \rightarrow \texttt{conditions / loops / rules} \rightarrow \texttt{output}$$

In a typical ML application:

$$\texttt{data} \rightarrow \texttt{training} \rightarrow \texttt{model} \rightarrow \texttt{prediction}$$

Transition for This Course

You already know how to write programs.
Now we will learn how to build programs that **learn from data**.

Typical AI/ML Applications

Typical AI/ML Applications

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

AI and Machine Learning are used whenever systems need to **recognize patterns, make predictions, or support decisions**.

- Classification
- Regression / Prediction
- Recommendation
- Anomaly Detection
- Computer Vision
- Natural Language Processing
- Forecasting
- Generative AI

Classification

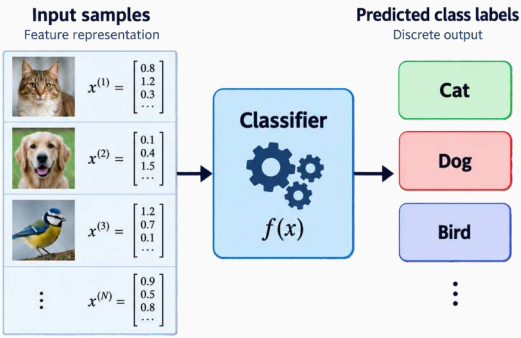
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Goal

Assign an input to one of several predefined categories.

Examples:

- Spam or non-spam email
- Fraudulent or legitimate transaction
- Disease present or absent
- Object category in an image



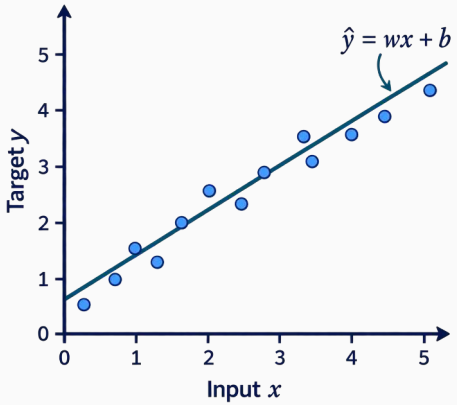
Regression and Prediction

Goal

Predict a continuous numerical value from input data.

Examples:

- Predicting house prices
- Estimating energy consumption
- Predicting machine lifetime
- Estimating travel time



Anomaly Detection

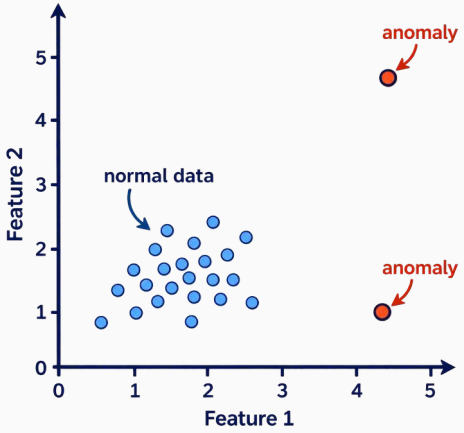
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Goal

Identify unusual observations that differ strongly from normal behavior.

Examples:

- Credit-card fraud
- Network intrusion detection
- Fault detection in machines
- Quality control in manufacturing



Computer Vision

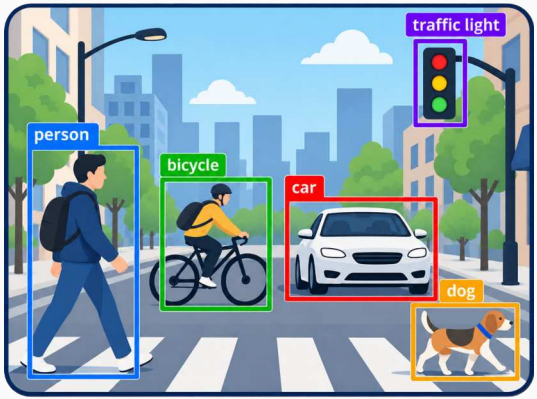
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Goal

Extract useful information from images and video.

Typical tasks include:

- Image classification
- Object detection
- Face recognition
- Image segmentation
- Visual inspection



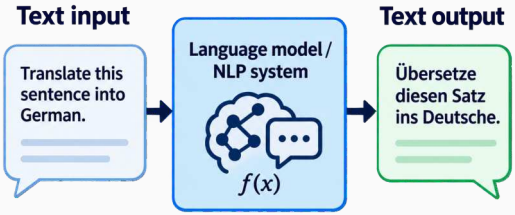
Natural Language Processing

Goal

Enable computers to process, understand, and generate human language.

Typical applications:

- Text classification
- Translation
- Summarization
- Question answering
- Conversational assistants



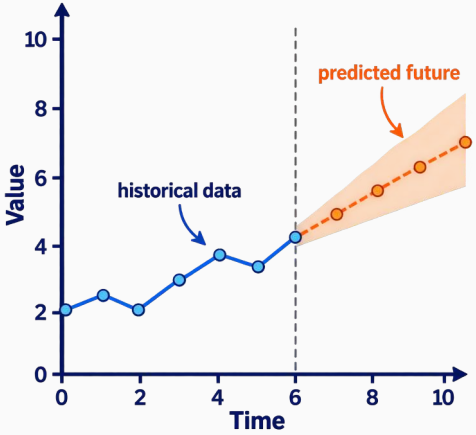
Forecasting

Goal

Use past observations to predict future values.

Examples:

- Sales forecasting
- Energy demand
- Traffic volume
- Weather-related quantities
- Production demand



Generative AI

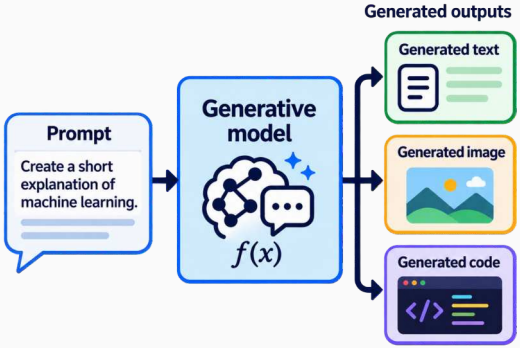
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Goal

Generate new content based on patterns learned from existing data.

Generative models can produce:

- Text
- Images
- Audio
- Video
- Program code



Different Applications, Similar Workflow

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Although the applications differ, many ML systems follow a similar pattern:

$$\text{Data} \rightarrow \text{Model} \rightarrow \text{Prediction} \rightarrow \text{Decision}$$

Examples

Email	→ Spam / Not Spam
Image	→ Object Class
Sensor Data	→ Machine Failure Risk
House Features	→ Estimated Price

Key Idea

Different AI applications often reduce to a small number of machine-learning problem types.

Importance of Datasets

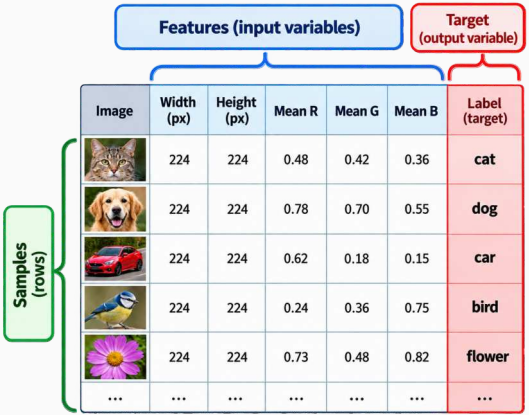
What is a Dataset?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A **dataset** is a structured collection of examples used to develop and evaluate a machine-learning system.

A dataset typically contains:

- **Samples / observations**
  Individual examples
- **Features**
  Information describing each sample
- **Target / label**
  The value we want to predict



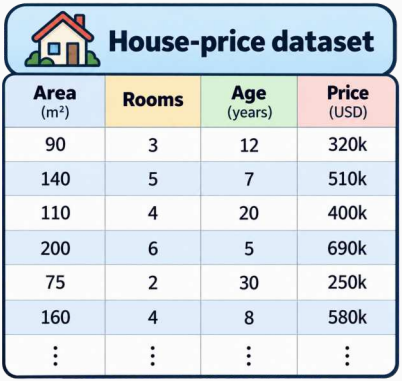
Example: A Simple Dataset

Suppose we want to predict the price of a house. Each row represents one house.
Possible features:

- Living area
- Number of rooms
- Age of the building
- Location

Target:

- House price



House-price dataset

Area (m^2)	Rooms	Age (years)	Price (USD)
90	3	12	320k
140	5	7	510k
110	4	20	400k
200	6	5	690k
75	2	30	250k
160	4	8	580k
⋮	⋮	⋮	⋮

Data is the Foundation of Machine Learning

Machine learning does not learn from the problem description. It learns from the **examples contained in the dataset**.

Therefore, model quality strongly depends on:

- how much data is available
- how representative it is
- how accurately it is measured
- how well labels are defined
- whether important cases are missing

Key Idea

A sophisticated model cannot compensate for fundamentally poor data.

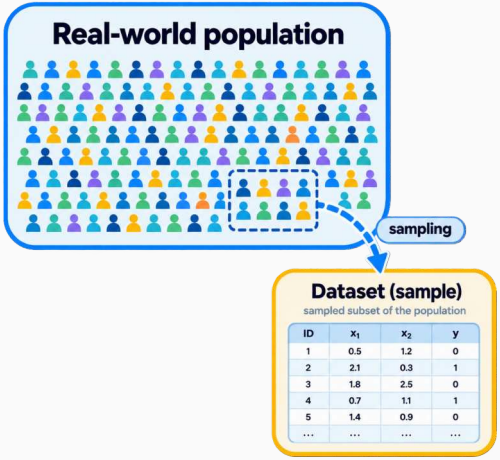
A Dataset is a Model of the Real World

A dataset is never the real world itself. It is a **sample** of the real world created through choices:

- What do we measure?
- Who or what do we observe?
- When do we collect data?
- What do we exclude?
- How do we define the labels?

Important

Biases or gaps in the dataset can become biases or gaps in the model.



Data Before Models

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A common beginner's view of ML:

Choose algorithm → train model

A more realistic view:

Problem → Data → Preparation → Model → Evaluation

Key Message

In practical machine learning, the dataset is often more important than the choice between two similar algorithms.

Learning Types and Typical Workflow

Three Types of Machine Learning

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Supervised Learning

- Learn from labeled examples
- Predict a known target
- Examples: classification, regression

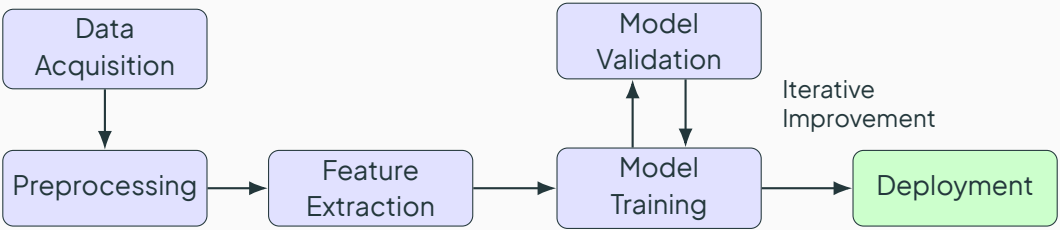
Unsupervised Learning

- Learn structure without target labels
- Discover groups or patterns
- Example: clustering

Reinforcement Learning

- Learn by interacting with an environment
- Actions are guided by rewards

Typical Machine Learning Workflow



Key Idea

This process forms the **Machine Learning Workflow**, which we will examine in detail in the next part of the course.

Python for Data Science

How Python Code Looks

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Java

```
int x = 10;

if (x > 5) {
    System.out.println("large");
}
```

Python

```
x = 10

if x > 5:
    print("large")
```

Key Difference

Python uses **indentation** to structure code instead of braces.

Java vs. Python: Basic Loop

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Java

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

- no semicolons
- no explicit type declaration needed
- blocks are defined by indentation

Python

```
for i in range(5):
    print(i)
```

Core Python Syntax We Need

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

- variables
- lists
- dictionaries
- conditions
- loops
- functions
- imports

We will learn Python only as far as needed for data analysis and machine learning.

```
values = [3, 5, 8]

person = {
    "name": "Ana",
    "age": 24
}

def square(x):
    return x * x

import pandas as pd
```

Python Collections

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

List

- ordered collection
- similar idea to an array or list in Java
- accessed by index

Dictionary

- key-value structure
- similar idea to a map in Java
- useful for structured information

```
numbers = [10, 20, 30]
print(numbers[0])

student = {
    "name": "Mia",
    "grade": 1.7
}

print(student["name"])
```

Functions and Imports

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Functions are defined with `def`.

Libraries are loaded with `import`.

In data science, we rarely work with only built-in Python.

We use specialized libraries for numerical computing, data tables, visualization, and machine learning.

```
def mean(a, b):
    return (a + b) / 2

result = mean(10, 20)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Python for Data Science

Important libraries

- `NumPy`: numerical arrays
- `pandas`: tabular data
- `Matplotlib`: visualization
- `scikit-learn`: classical machine learning
- `PyTorch`: deep learning and neural networks
- `Ultralytics`: YOLO for object detection



Why Jupyter Notebooks?

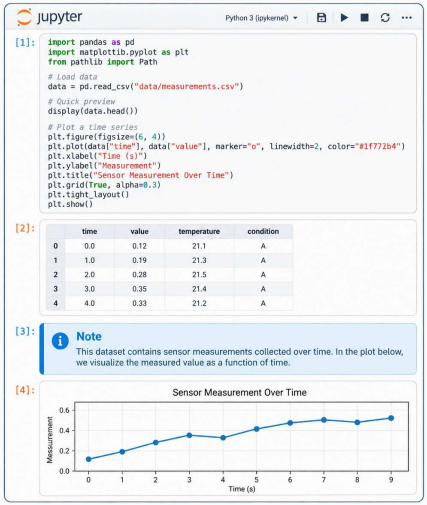
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A Jupyter Notebook combines:

- executable code
- output and results
- plots and tables
- explanatory text

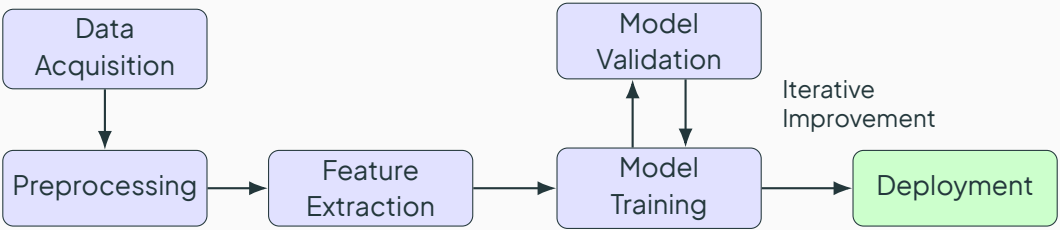
This makes notebooks useful for:

- experimenting with data
- documenting decisions
- explaining results
- sharing ML workflows



The Machine Learning Workflow

The Machine Learning Workflow



Important

Machine learning is not only about training a model. It is an **end-to-end, iterative workflow** from problem definition and data to evaluation and application.

Step 1: Define the Problem

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Before choosing a model, we need to define:

- **What is the task?**
- **What should be predicted?**
- **What information is available?**
- **How will success be measured?**

Key Idea

A poorly defined problem cannot be fixed by a better algorithm.

From a Real-World Question to an ML Task

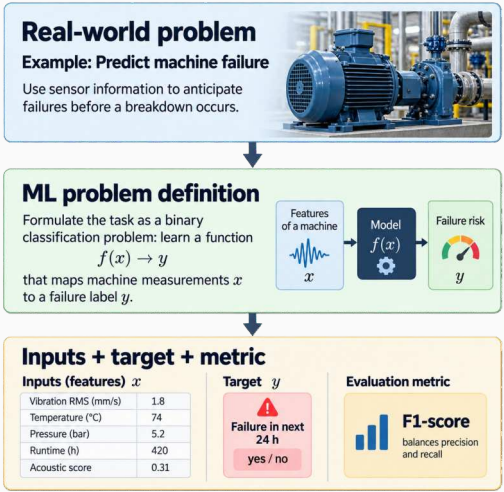
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Real-world question:

Which machines are likely to fail soon?

Translate this into:

- **Input:** sensor measurements
- **Target:** failure / no failure
- **Task:** classification
- **Output:** failure probability or class



What Are the Inputs and the Target?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Most supervised ML problems can be expressed as:

Input features $\rightarrow$ Target

For each sample:

- **Features** describe the observation
- **Target** is what we want to predict

Example:

Area, Rooms, Age, Location $\rightarrow$ House Price



Classification or Regression?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Classification

Predict a **category**.

- Spam / Not Spam
- Fault / No Fault
- Cat / Dog / Bird

Input → Class

Regression

Predict a **continuous value**.

- House price
- Energy consumption
- Remaining lifetime

Input → Number

First Question to Ask

Is the desired output a **class** or a **numerical value**?

What Does “Good” Mean?

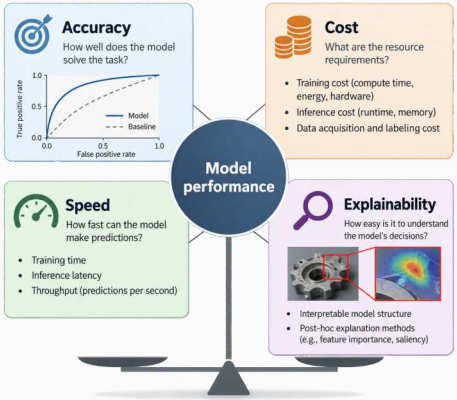
Define how model quality will be judged.

Possible questions:

- How accurate must predictions be?
- Are all mistakes equally costly?
- Is false positive or false negative worse?
- Is prediction speed important?
- Must the model be explainable?

Success Criterion

The evaluation method should reflect the actual goal of the application.



A Good Problem Definition

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A useful ML problem should clearly state:

1. Input data
2. Target or desired output
3. Type of task
4. Evaluation criterion
5. Intended use of the prediction

Important

"Use AI to improve the system" is not a machine-learning problem definition.



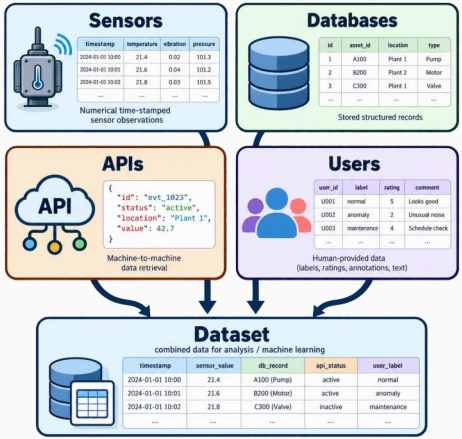
Step 2: Acquire and Understand the Data

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Once the problem is defined, we need suitable data.

Typical questions are:

- Where does the data come from?
- Do we have enough data?
- Is it representative?
- Are measurements reliable?
- Are labels available?



Dataset Quality Matters

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A useful dataset should be:

- **Relevant**
  Contains information related to the task
- **Representative**
  Reflects the cases the model will encounter
- **Accurate**
  Measurements and labels are trustworthy
- **Sufficient**
  Contains enough examples
- **Consistent**
  Similar variables are measured in the same way

Data Collection Can Be the Largest Part of the Work

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Training a model may take minutes or hours. Building a useful dataset may take **days, months, or even years**.

Typical effort includes:

- deciding what to measure
- collecting enough examples
- cleaning corrupted records
- labeling data
- integrating multiple sources
- checking data quality
- handling privacy and permissions

Is the Data Representative?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

The training data should resemble the data the model will encounter later.

Problems arise when:

- important cases are missing
- some groups are overrepresented
- the data was collected under unusual conditions
- the real-world environment changes over time

Example

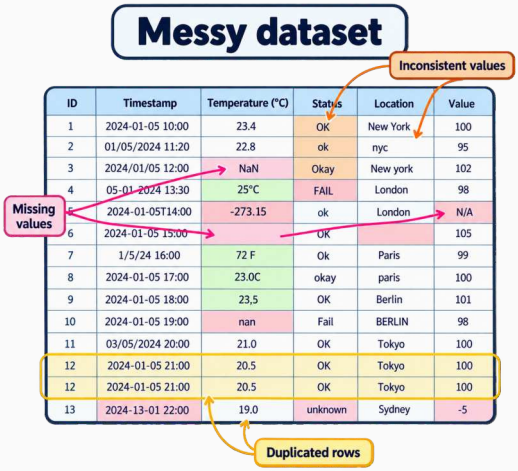
A model trained only on daytime images may perform poorly at night.

Data Quality Problems

Common issues include:

- missing values
- incorrect measurements
- duplicated records
- inconsistent formats
- mislabeled samples
- outliers and noise

These problems may affect the model more than the choice of algorithm.

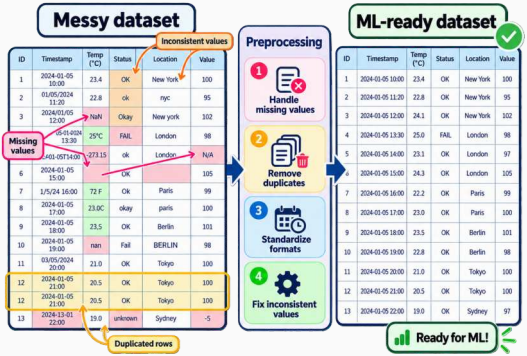


ID	Timestamp	Temperature (°C)	Status	Location	Value
1	2024-01-05 10:00	23.4	OK	New York	100
2	01/05/2024 11:20	22.8	ok	nyc	95
3	2024/01/05 12:00	NaN	Okay	New york	102
4	05-01-2024 13:30	25°C	FAIL	London	98
5	2024-01-05T14:00	-273.15	ok	London	N/A
6	2024-01-05 15:00		OK		105
7	1/5/24 16:00	72 F	Ok	Paris	99
8	2024-01-05 17:00	23.0C	okay	paris	100
9	2024-01-05 18:00	23,5	OK	Berlin	101
10	2024-01-05 19:00	nan	Fail	BERLIN	98
11	03/05/2024 20:00	21.0	OK	Tokyo	100
12	2024-01-05 21:00	20.5	OK	Tokyo	100
12	2024-01-05 21:00	20.5	OK	Tokyo	100
13	2024-13-01 22:00	19.0	unknown	Sydney	-5

Step 3: Preprocess the Data

Typical preprocessing tasks include:

- handling missing values
- correcting inconsistent data
- encoding categorical variables
- scaling numerical features
- selecting or creating useful features



Common Preprocessing Operations

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Missing values

- remove incomplete samples
- replace values using mean, median, etc.

Categorical data

- convert categories into numerical representations

Numerical data

- scaling or normalization may be required

Example: Encoding Categorical Data

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Many ML algorithms require numerical input.

Suppose the dataset contains:

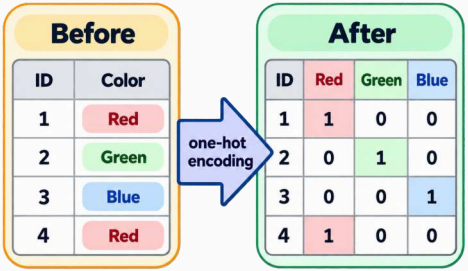
```
Color = {Red, Green, Blue}
```

A common approach is **one-hot encoding**:

$$\text{Red} \rightarrow [1, 0, 0]$$

$$\text{Green} \rightarrow [0, 1, 0]$$

$$\text{Blue} \rightarrow [0, 0, 1]$$



Feature Extraction and Engineering

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Sometimes raw measurements are not the most useful inputs. We may create a new **feature** that better describes the problem.

Example:

$$\text{Distance}, \text{Time}$$

can be transformed into

$$\text{Average Speed} = \frac{\text{Distance}}{\text{Time}}$$

Key Idea

Feature engineering uses knowledge about the problem to create useful model inputs.

Preprocessing is Part of the Model Pipeline

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Preprocessing should be performed in a **consistent and reproducible way**.

$$\text{Raw Data} \rightarrow \text{Preprocessing} \rightarrow \text{Features} \rightarrow \text{Model}$$

The same transformations used during training must also be applied to new data later.

Important

Preprocessing must be designed carefully to avoid introducing information that should not be available during training.

Splitting the Data

Why Split the Data?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A model should not only perform well on the data it has seen.

We want to know how well it works on **new, unseen data**.

Therefore, the dataset is divided into separate parts:

- training data
- validation data
- test data

Training, Validation, and Test Datasets

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Training Set

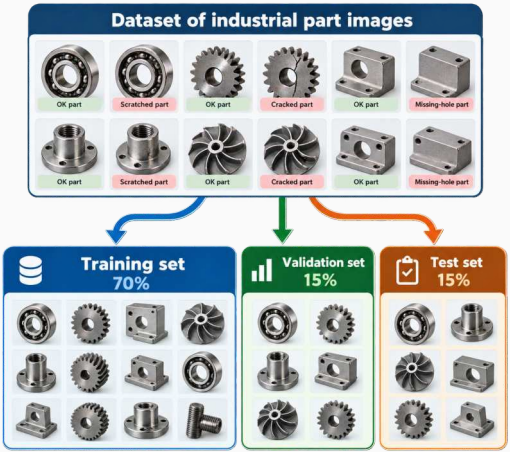
- used to fit the model

Validation Set

- used to compare models
- used to tune settings and hyperparameters

Test Set

- used only for final evaluation
- estimates performance on unseen data



What Happens if We Test on the Training Data?

Suppose a model achieves:

$$\text{Training Accuracy} = 99\%$$

Does this mean the model is good?

Not necessarily.

The model may simply have adapted too closely to the training examples.

Problem

Training performance tells us how well the model fits known data, not how well it generalizes.

Generalization

The goal of machine learning is not to memorize examples.

A good model should learn patterns that also work for **unseen samples**.

This ability is called:

Generalization

Successful Learning
Good performance on both known and unseen data.

Avoid Data Leakage

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

The test set must remain independent from training.

Data leakage occurs when information from validation or test data influences the training process.

Examples:

- preprocessing before splitting
- selecting features using all data
- repeatedly tuning based on test results

The Workflow So Far

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

We now have everything required before learning begins:

1. Define the problem
2. Acquire and understand the data
3. Split the dataset
4. Preprocess the data

$$\text{Problem} \rightarrow \text{Data} \rightarrow \text{Split} \rightarrow \text{Preprocessing}$$

Next Lecture

How does a model actually **learn** from the training data?

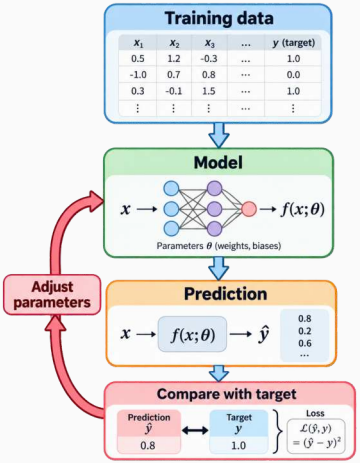
How Machine Learning Learn?

What Does “Learning” Mean?

A machine-learning model contains adjustable **parameters**.

During training, the model repeatably:

1. receives training examples,
2. makes predictions,
3. compares predictions with the correct values,
4. adjusts its parameters to reduce the error.



A Simple Model: Linear Regression

Suppose we want to predict a numerical value from one input feature. A simple model is:

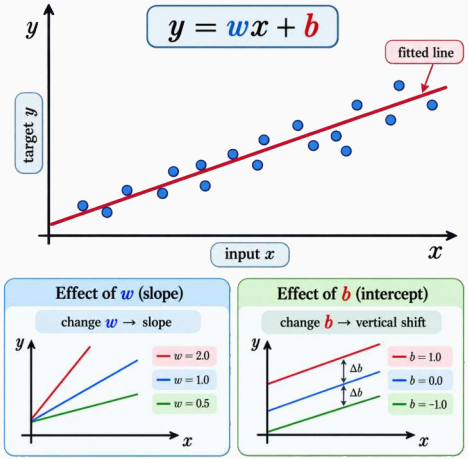
$$\hat{y} = wx + b$$

where:

- x: input feature
- $\hat{y}$: predicted value
- w: weight / slope
- b: bias / intercept

Learning

Training means finding suitable values for w and b from the data.



Which Model is Better?

Different choices of w and b produce different predictions:

$$\hat{y} = wx + b$$

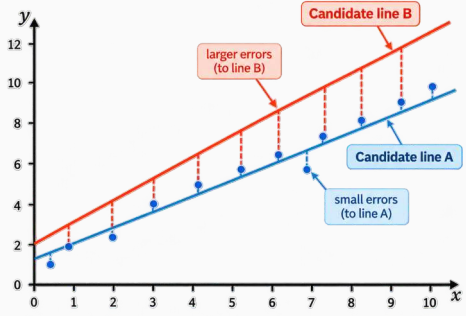
For each training sample, we compare:

Prediction $\hat{y}$ with Target y

The difference is the prediction error:

$$e = y - \hat{y}$$

To compare models, we summarize these errors using a **loss function**.



Loss Function

A common choice for regression is the **Mean Squared Error (MSE)**:

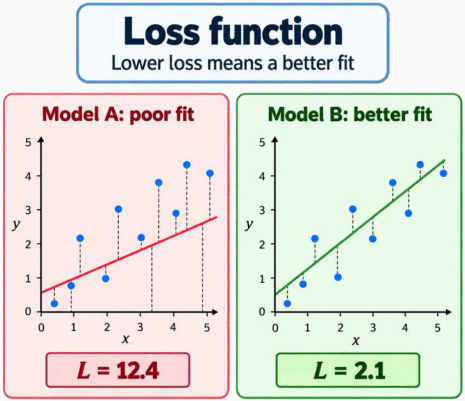
$$L = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

The loss converts model performance into a single number.

- large prediction errors → large loss
- small prediction errors → small loss

Goal of Training

Find w and b that make the loss as small as possible.



Training as an Optimization Problem

For linear regression:

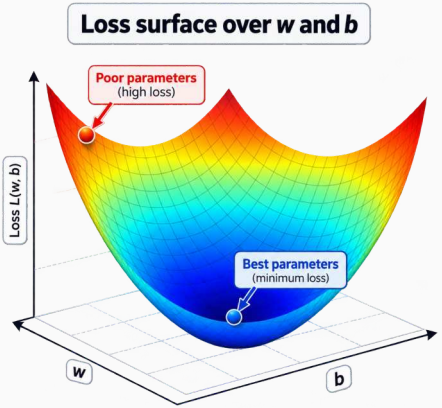
$$\hat{y} = wx + b$$

the loss depends on the parameters w and b:

$$L = L(w, b)$$

Training means searching for parameter values that minimize the loss:

$$(w^*, b^*) = \arg\min_{w,b} L(w, b)$$



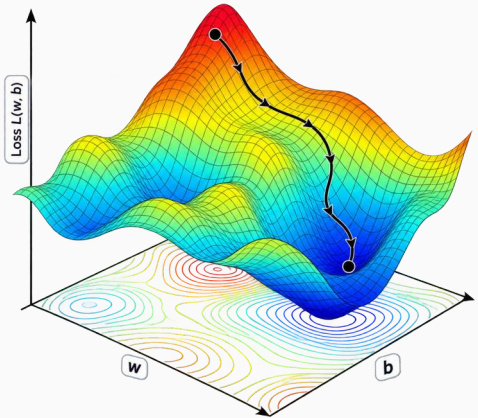
How Does the Model Improve?

The model starts with some initial parameter values. Then it repeatedly:

1. makes predictions,
2. calculates the loss,
3. determines how parameters should change,
4. updates the parameters.

This process continues until the loss is sufficiently small or training is stopped.

Prediction → Loss → Update → Repeat



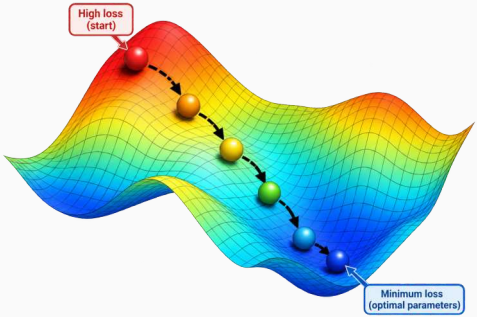
Gradient Descent

Gradient descent is an algorithm for minimizing the loss. Basic idea:

1. Calculate the current loss
2. Determine the direction of steepest increase
3. Move in the opposite direction

Intuition

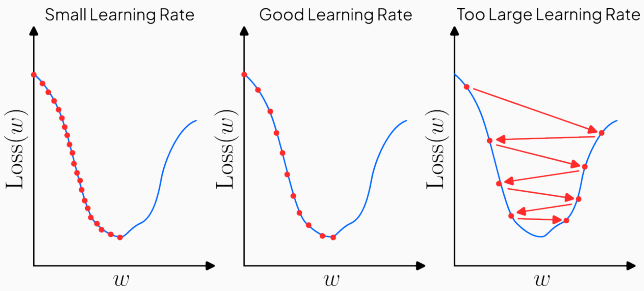
Like walking downhill: at each step, move in the direction that reduces the loss.



Learning Rate

The **learning rate** controls how large each update step is.

- Too small: learning is very slow
- Too large: the model may overshoot the minimum
- Suitable value: stable progress toward lower loss



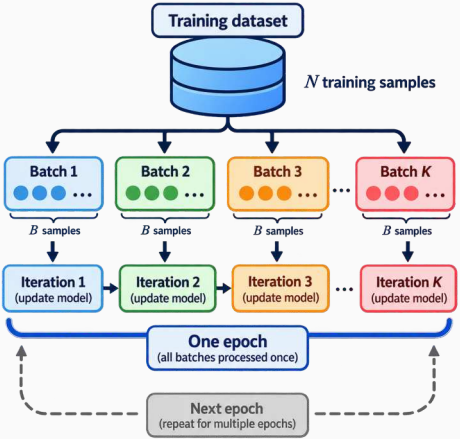
Epochs, Batches, and Iterations

Large datasets are often processed in smaller groups called **batches**.

- **Batch**: subset of the training data
- **Iteration**: one parameter update
- **Epoch**: one full pass through all training data

Why Batches?

They make training more efficient and practical for large datasets.



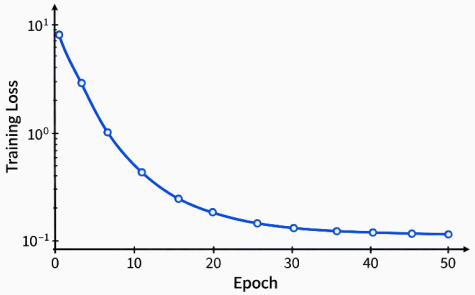
Training Curves

During training, we can track how the loss changes over time. A typical successful training process shows:

- high loss at the beginning,
- decreasing loss over several epochs,
- gradual stabilization.

Interpretation

A decreasing training loss indicates that the model is learning to fit the training data better.



Underfitting and Overfitting

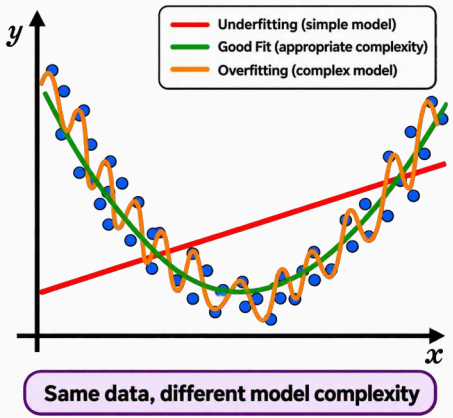
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Underfitting

- model is too simple
- important patterns are not captured
- poor performance on training and unseen data

Overfitting

- model fits the training data too closely
- noise is learned as if it were a real pattern
- training performance is high, but generalization is poor



Training vs. Validation Performance

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A low training loss does not always mean that the model is good. We therefore compare:

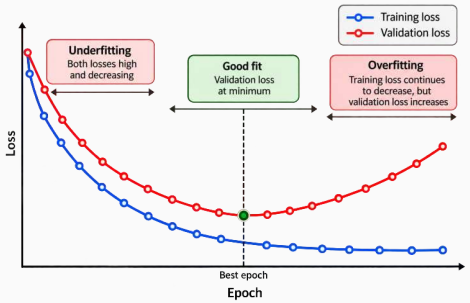
- **Training loss:** performance on known data
- **Validation loss:** performance on unseen validation data

Good Sign

Both training and validation loss decrease and then stabilize.

Warning Sign

Training loss decreases, but validation loss increases.



Parameters vs. Hyperparameters

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Model Parameters

- learned during the training from the data
- determine the model's predictions

Example for linear regression:

$$\hat{y} = wx + b$$

$w,\ b$ are learned parameters

Hyperparameters

- chosen before or during model development
- control how the model learns or how complex it is

Examples:

- learning rate
- number of epochs
- batch size
- model complexity

When Does Training Stop?

Training cannot continue forever. Common stopping criteria are:

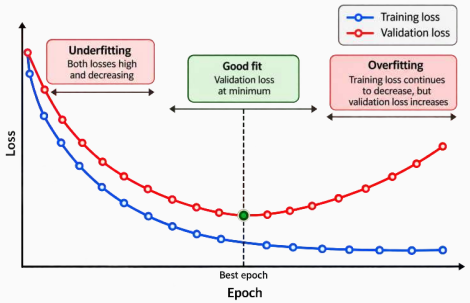
- fixed number of epochs
- loss changes only very little
- validation performance stops improving

A common strategy is **early stopping**:

- monitor validation loss
- stop training when it no longer improves

Goal

Stop after the model has learned useful patterns, but before it starts to overfit.



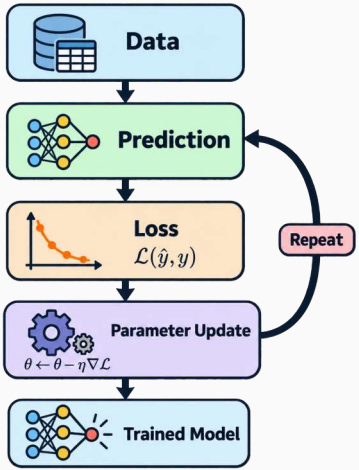
How Machine Learning Learns

The basic learning process can now be summarized as:

1. Make a prediction
2. Compare with the target
3. Calculate the loss
4. Adjust model parameters
5. Repeat over many examples
6. Monitor validation performance

Core Idea

Machine learning is an iterative search for model parameters that produce good predictions on **unseen data**.



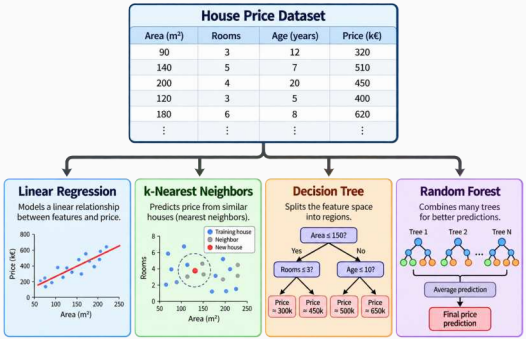
Classical ML Methods

From Training to Model Choice

In the previous lecture, we saw how a model learns based on preprocessed dataset. But there is not only one type of model.

Different model families learn patterns in different ways:

- mathematical relationships
- similarity between samples
- decision rules
- combinations of many models



Linear Regression

Linear regression models a numerical target as a linear combination of the input features.
For one feature:

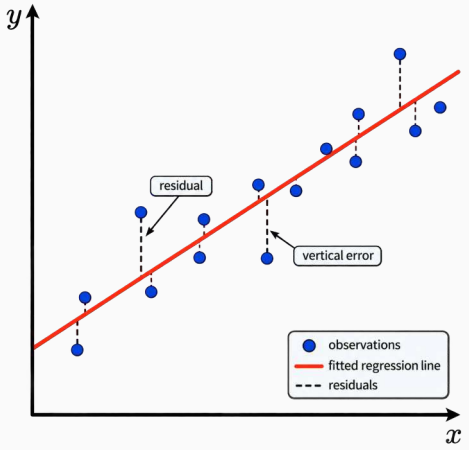
$$\hat{y} = wx + b$$

The model learns:

- w: how strongly x influences the prediction
- b: the intercept

Typical Use

Predicting continuous values such as prices, temperatures, or energy consumption.



Linear Regression: Strengths and Limits

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Strengths

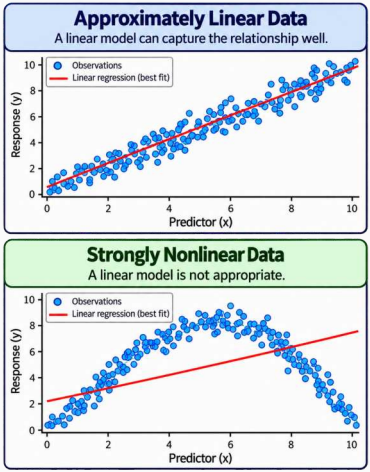
- simple and fast
- easy to interpret
- useful as a baseline model

Limitations

- assumes a linear relationship
- may perform poorly on complex patterns
- can be sensitive to outliers

Key Idea

Linear regression is simple, but simplicity can be an advantage when the relationship is approximately linear.

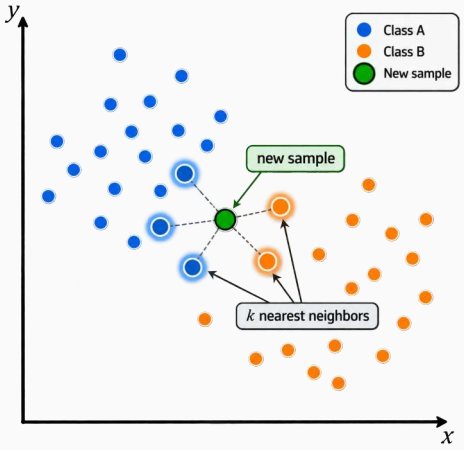


k-Nearest Neighbors (k-NN)

k-NN makes predictions based on the **most similar training samples**.

For a new sample:

1. measure its distance to the training samples,
2. select the k nearest neighbors,
3. the new sample belongs to the majority of the nearest neighbors selected.



The Choice of k

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

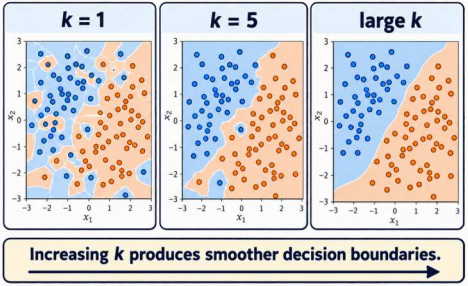
The parameter k controls how many neighbors influence the prediction.

Small k

- very sensitive to individual samples
- can create complex decision boundaries
- higher risk of overfitting

Large k

- smoother predictions
- less sensitive to noise
- may underfit the data



k-NN: Strengths and Limits

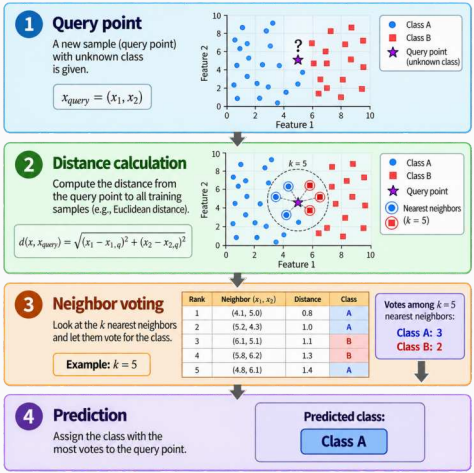
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Strengths

- simple and intuitive
- almost no explicit training
- can model nonlinear decision boundaries

Limitations

- predictions can be slow for large datasets
- sensitive to feature scaling
- performance depends strongly on k
- distance becomes less meaningful with many features



Decision Trees

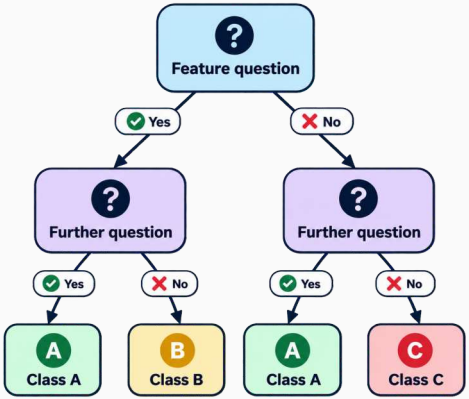
A decision tree makes predictions by asking a sequence of **feature-based questions**.

Example:

$$\text{Is temperature} > 25°\text{C?}$$

Depending on the answer, the sample follows a different branch.
A tree consists of:

- **Root node**: first decision
- **Internal nodes**: further decisions
- **Branches**: outcomes of decisions
- **Leaves**: final predictions



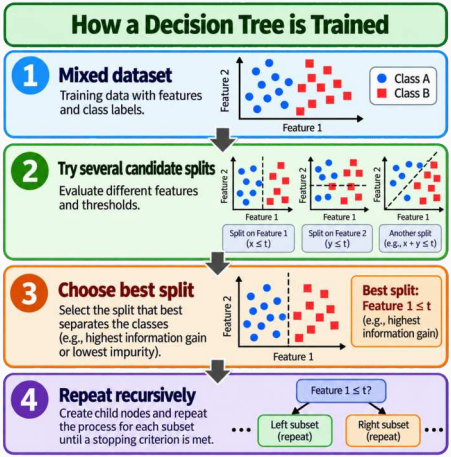
How Is a Decision Tree Trained?

Training means finding useful questions that split the data.

At each node, the algorithm:

1. considers possible features and split values,
2. divides the training samples,
3. measures how well the split separates the target classes,
4. chooses the best split,
5. repeats the process for each branch.

For classification, the goal is to create groups that are as **pure** as possible.



What Makes a Good Split?

A good split separates the data into groups that contain mostly one class.

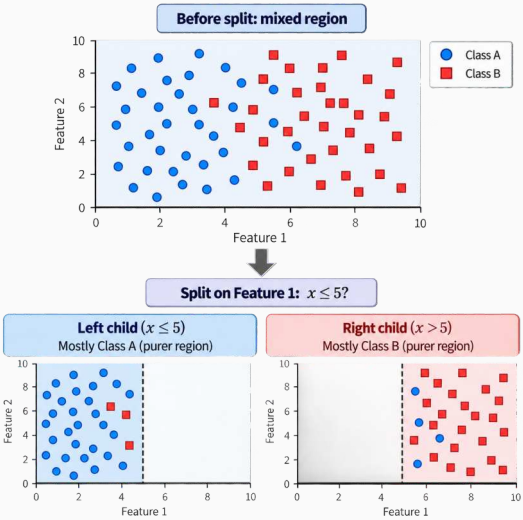
Example:

Before split: ●●●●●●

After good split: ●●● ●●●

Common measures include:

- Gini impurity
- entropy / information gain



Decision Trees: Strengths and Limits

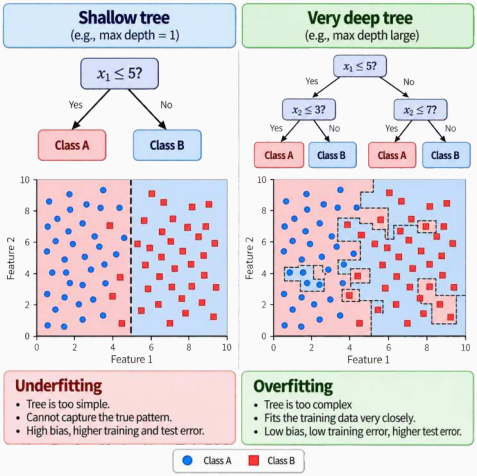
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Strengths

- easy to visualize and interpret
- handles nonlinear relationships
- little preprocessing required
- works with numerical and categorical features

Limitations

- deep trees can overfit easily
- small data changes may produce a different tree
- a single tree may have limited predictive performance



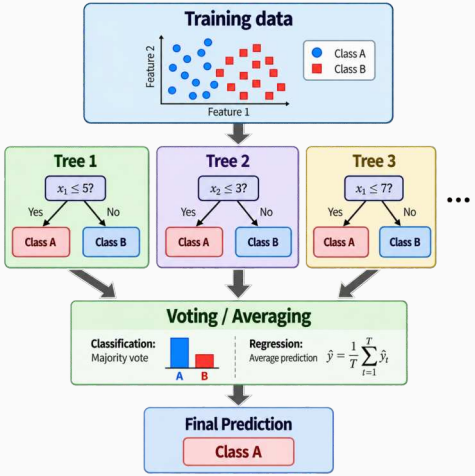
Random Forests

A random forest combines many decision trees into one model.

Each tree is trained on:

- a different random sample of the training data
- a random subset of the available features

The predictions of all trees are then combined based on the majority of the decisions or the average of the values predicted.



Why Use Many Trees?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A single decision tree can be unstable and may overfit.

Random forests reduce this problem by combining **many different trees**.

If individual trees make different errors, averaging or voting can produce a more reliable result.

Key Idea
Many diverse models can work better together than one model alone.

How Is a Random Forest Trained?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

For each tree:

1. randomly sample training examples,
2. grow a decision tree,
3. consider only a random subset of features at each split,
4. repeat to create many trees.

The individual trees are therefore similar, but not identical.

Random Forests: Strengths and Limits

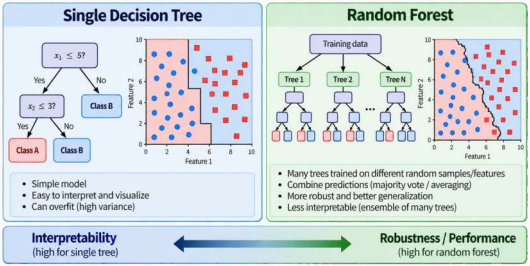
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Strengths

- often strong performance on tabular data
- handles nonlinear relationships
- less prone to overfitting than one deep tree
- relatively little preprocessing required

Limitations

- less interpretable than a single tree
- requires more memory and computation
- large forests can be slower to predict



Model Evaluation

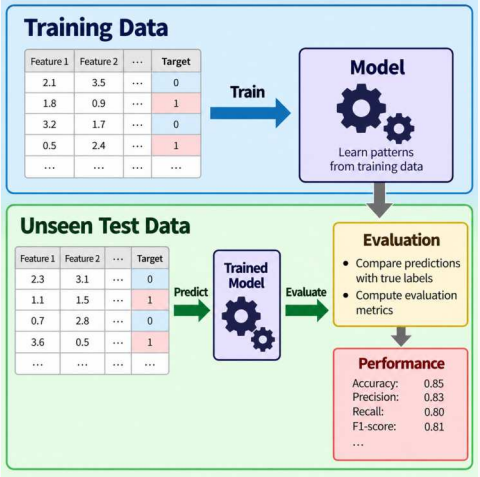
Why Model Evaluation Matters

A model can fit the training data very well and still perform poorly on new data.
Model evaluation helps us answer:

- How well does the model generalize?
- Is one model better than another?
- Is the model good enough for the application?
- What kinds of errors does it make?

Key Idea

The goal is not to achieve the best training score, but to perform well on **unseen data**.



Evaluation Must Match the Task

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Different machine-learning problems require different evaluation methods.

Regression

- How far are predictions from the true numerical values?

Classification

- How often are classes predicted correctly?
- What kinds of mistakes are made?

Important

There is no single metric that is suitable for every ML problem.

Evaluating Regression Models

For regression, predictions are numerical values.
For each sample, the prediction error is:

$$e_i = y_i - \hat{y}_i$$

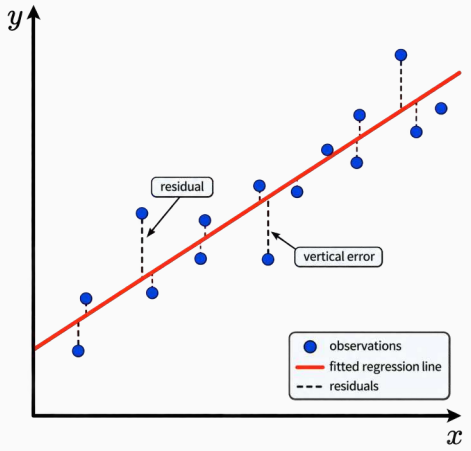
where:

- y_i: true value
- $\hat{y}_i$: predicted value

We summarize these errors over the complete test set using an **evaluation metric**.

Goal

Smaller prediction errors indicate better regression performance.



MAE, MSE, and RMSE

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

- average absolute prediction error
- easy to interpret

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- penalizes large errors more strongly

Root Mean Squared Error

$$\text{RMSE} = \sqrt{\text{MSE}}$$

RMSE is expressed in the same unit as the target variable.

Interpretation

- MAE: average error size
- RMSE: gives more weight to large errors

Classification Evaluation

For classification, predictions are class labels.
Example:

Email → Spam / Not Spam

Evaluation asks:

- Which samples were classified correctly?
- Which samples were misclassified?
- What kinds of mistakes were made?

Key Idea

For classification, the type of error often matters as much as the number of errors.

Sample	True label	Predicted label	
1	A	A	✓
2	B	B	✓
3	A	B	✗
4	B	B	✓
5	A	A	✓
6	B	A	✗
7	A	A	✓
8	B	B	✓

Confusion Matrix

A confusion matrix compares:

- the **true class**
- the **predicted class**

For binary classification:

- **TP**: correctly predicted positive
- **TN**: correctly predicted negative
- **FP**: predicted positive, but actually negative
- **FN**: predicted negative, but actually positive

	Pred. +	**Pred. –**
True +	TP	FN
True –	FP	TN

Question

Are false positives and false negatives equally serious?

Accuracy

Accuracy measures the fraction of correct predictions.

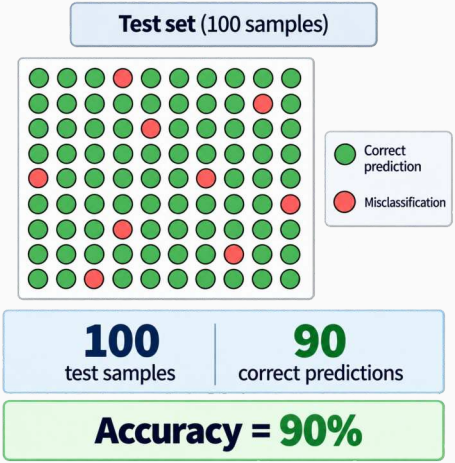
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

It is easy to understand:

How often is the model correct?

Limitation

Accuracy can be misleading when classes are imbalanced.



Precision and Recall

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Among all samples predicted as positive:

How many were truly positive?

High precision means few false positives.

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

Among all truly positive samples:

How many did we find?

High recall means few false negatives.

Practical Meaning

Precision focuses on the reliability of positive predictions; recall focuses on finding as many positives as possible.

Imbalanced Datasets

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A dataset is **imbalanced** when one class appears much more often than another.

Example:

- 990 normal transactions
- 10 fraudulent transactions

A model that always predicts “normal” would achieve:

$$\frac{990}{1000} = 99\% \text{ accuracy}$$

Problem

High accuracy can hide poor performance on the important minority class.

Why Accuracy Can Be Misleading

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Consider rare disease detection:

- 1000 patients
- 20 actually have the disease
- model predicts “no disease” for everyone

Accuracy:

$$\frac{980}{1000} = 98\%$$

But recall for the disease class is:

$$\frac{0}{20} = 0\%$$

Key Message

Always inspect which class the model is getting wrong.

Why Cross-Validation?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

A single train/validation split may give a misleading result.

The measured performance can depend on which samples happen to be in the validation set.

Cross-validation reduces this problem by evaluating the model on several different data splits.

Goal
Obtain a more reliable estimate of how well the model generalizes to unseen data.

k-Fold Cross-Validation

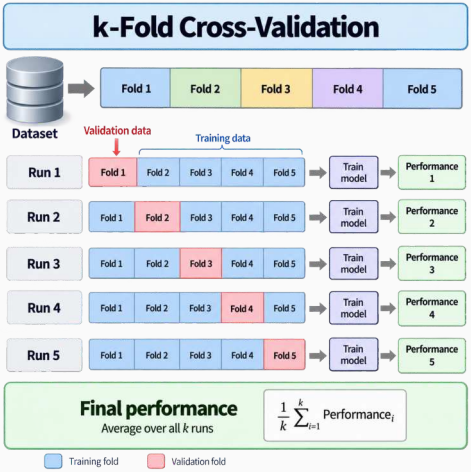
In k**-fold cross-validation**:

1. Split the training data into k folds
2. Train on $k-1$ folds
3. Validate on the remaining fold
4. Repeat until every fold has been used for validation

The final score is usually the average:

$$\text{CV Score} = \frac{1}{k}\sum_{i=1}^{k} \text{Score}_i$$

Common choices are $k = 5$ or $k = 10$.



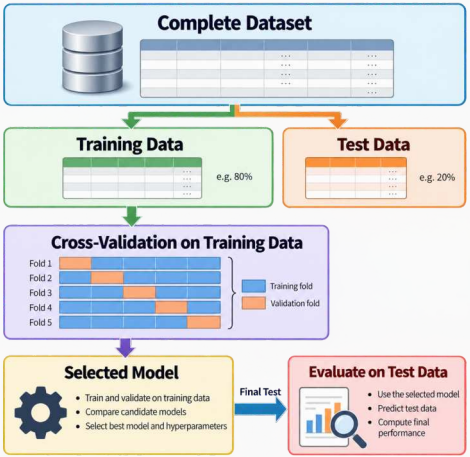
Cross-Validation and the Test Set

Cross-validation is usually performed on the **training data**.
It can be used to:

- compare different models
- select hyperparameters
- estimate model stability

After model selection:

evaluate once on the independent test set



Choosing the Final Model

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

The model with the highest score is not always the best choice. Always consider:

- **Predictive performance**
- **Generalization** to unseen data
- **Interpretability**
- **Training and prediction speed**
- **Model complexity**
- **Robustness** to noise and data changes

Key Idea

Model selection is a trade-off between performance and practical requirements.

Neural Networks and Deep Learning

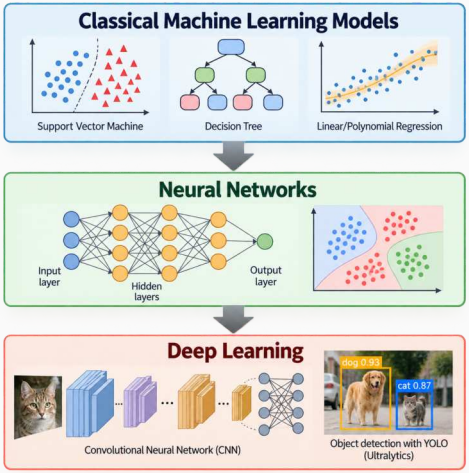
From Classical ML to Neural Networks

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Classical machine-learning models can work very well, especially for structured tabular data.

Neural networks are another model family, often useful for more complex data:

- images
- text
- audio
- high-dimensional sensor data



The Artificial Neuron

An artificial neuron receives several input values:

$$x_1, x_2, \ldots, x_n$$

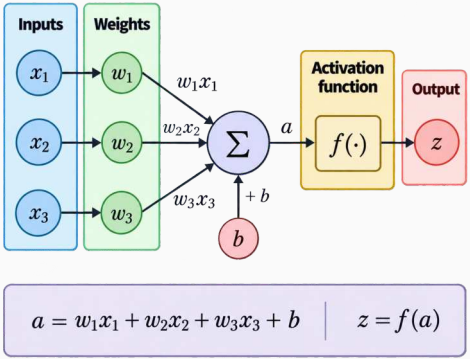
Each input has a corresponding weight:

$$w_1, w_2, \ldots, w_n$$

The neuron computes a weighted sum:

$$a = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

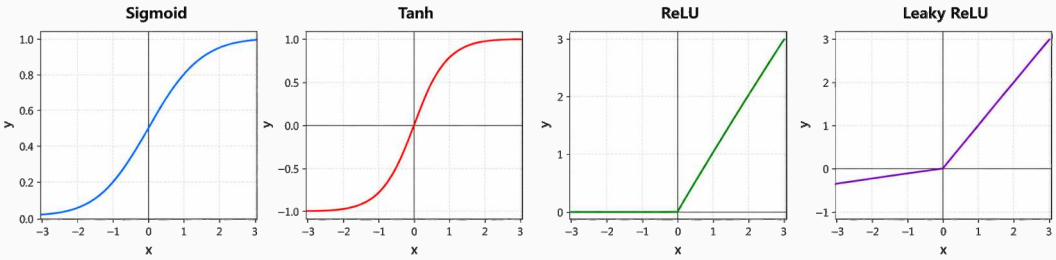
$$z = f(a)$$



Activation Functions

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Activation functions introduce non-linearity before the weighted sum is passed to the output.



From Neurons to a Neural Network

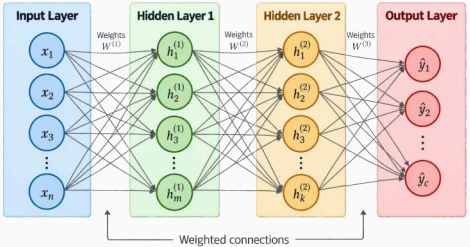
A neural network connects many artificial neurons in layers.
Typical structure:

- **Input layer**: receives the features
- **Hidden layers**: transform the information
- **Output layer**: produces the prediction

Each connection has a weight that is adjusted during training.

Key Idea

A neural network learns a sequence of transformations from input data to prediction.



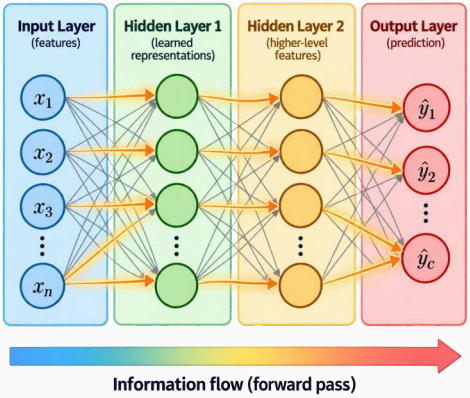
The Forward Pass

During a **forward pass**, the input data moves through the network from layer to layer.
Each layer:

1. receives values from the previous layer,
2. applies weights and biases,
3. applies an activation function,
4. passes the result to the next layer.

At the output layer, the network produces a prediction:

$$x \rightarrow \text{Hidden Layers} \rightarrow \hat{y}$$



From Prediction to Error

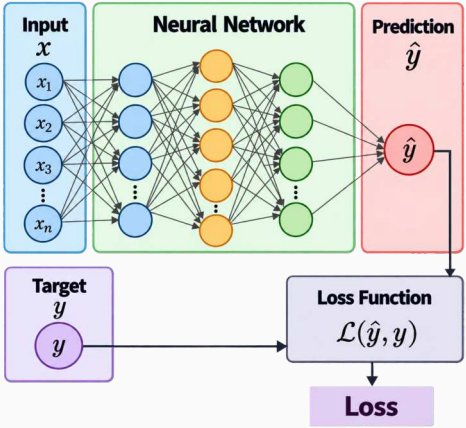
After the forward pass, the prediction is compared with the true target.

$$\hat{y} \quad \text{vs.} \quad y$$

A **loss function** measures how wrong the prediction is:

$$L(y, \hat{y})$$

The training objective is to adjust the model parameters to reduce the loss.

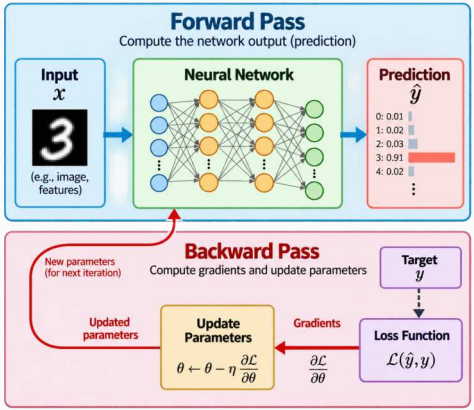


Backpropagation

A neural network may contain thousands or millions of weights.
Backpropagation determines how each parameter contributed to the loss.
The process works backwards through the network:

$$\text{Loss} \rightarrow \text{Output Layer} \rightarrow \text{Hidden Layers} \rightarrow \text{Earlier Layers}$$

The calculated gradients are then used by an optimizer, such as gradient descent, to update the weights.



Training a Neural Network

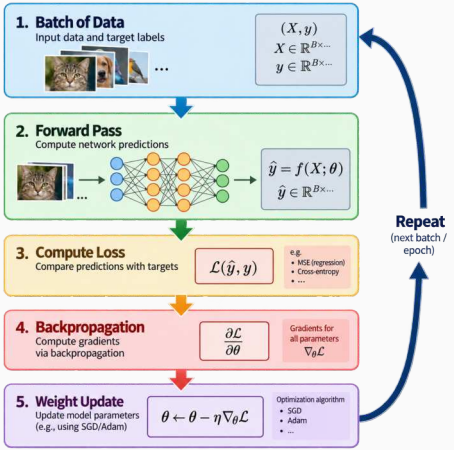
ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Neural-network training repeatedly combines the steps we have already seen.

For each batch:

1. perform a forward pass,
2. calculate the loss,
3. run backpropagation,
4. update the weights with an optimizer.

This process is repeated over many batches and epochs.



Training and Validation Curves

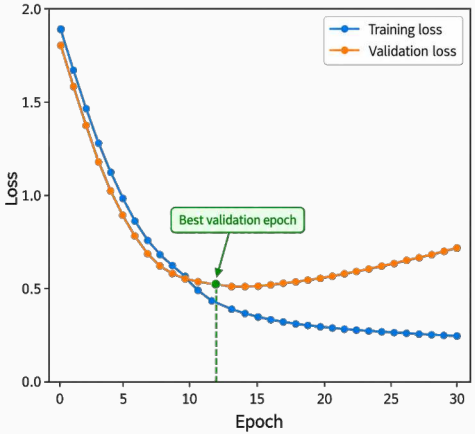
During training, we typically monitor:

- **training loss**
- **validation loss**

and stop if the model overfits.

A healthy training process often shows:

- both losses decreasing initially,
- validation loss reaching a minimum,
- training loss possibly continuing to decrease.



What is Deep Learning?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

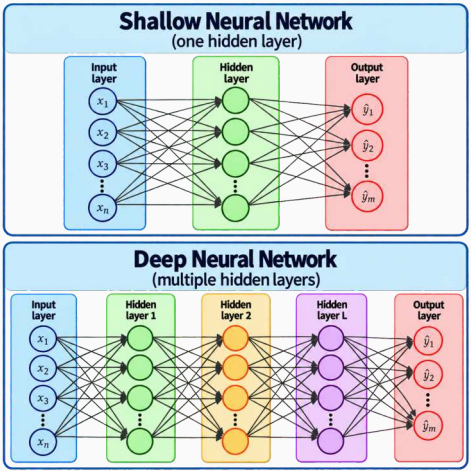
Deep Learning uses neural networks with multiple hidden layers.
Each layer transforms the representation of the data:

$$\text{Input} \rightarrow \text{Layer 1} \rightarrow \text{Layer 2} \rightarrow \cdots \rightarrow \text{Output}$$

Deeper networks can learn increasingly complex patterns.

Key Idea

"Deep" refers to the number of successive layers used to transform the input.



Learning Hierarchical Features

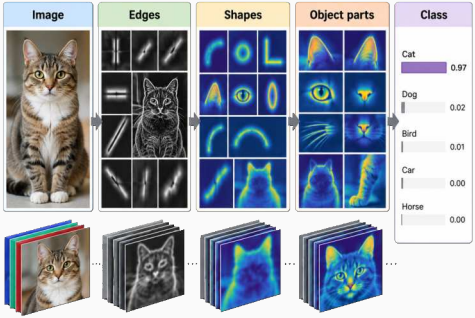
One strength of deep learning is that useful features can be learned automatically from raw data.
For image recognition, for example:

$$\text{Pixels} \rightarrow \text{Edges} \rightarrow \text{Shapes} \rightarrow \text{Objects}$$

Earlier layers may learn simple patterns, while deeper layers combine them into more complex representations.

Difference from Manual Feature Engineering

Deep networks can learn both the **features** and the **prediction model** during training.



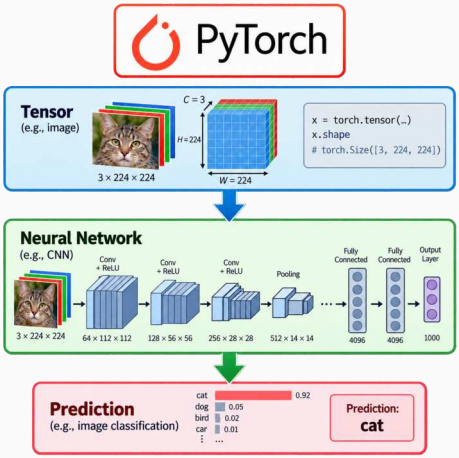
PyTorch

PyTorch is a framework for building and training neural networks.

- `Tensor`: numerical data
- `nn.Module`: neural network
- Loss function: prediction error
- Optimizer: update weights
- Automatic differentiation

Connection to This Lecture

Forward Pass → Loss → Backpropagation → Update



A Neural Network in PyTorch

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

```
import torch
from torch import nn

model = nn.Sequential(
    nn.Linear(4, 16),
    nn.ReLU(),
    nn.Linear(16, 3)
)
```

What do we see?

- 4 input features
- 16 hidden neurons
- ReLU activation
- 3 output values

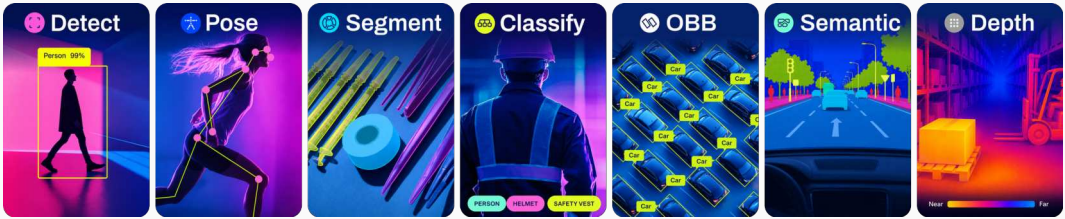
Key Idea

The architecture we drew earlier can be expressed directly in code.

Ultralytics and YOLO

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

Ultralytics provides pretrained **YOLO** deep-learning models for computer vision.



YOLO in a Few Lines of Python

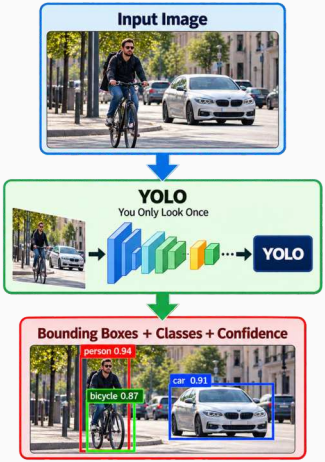
```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

results = model("image.jpg")

results[0].show()
```

A pretrained model can immediately perform inference on new images.



PyTorch or Ultralytics?

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

PyTorch

- build your own network
- control the training process
- understand deep learning
- flexible for many domains

Ultralytics / YOLO

- pretrained vision models
- fast project prototyping
- object detection and related tasks
- fine-tune on your own images

For Your Project

Use the simplest tool that solves the problem: classical ML, your own neural network, or a pretrained model.

Group Project

Group Project Overview

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

- Teams of **three students**
- Self-defined project idea
- Project must involve a **webcam**
- Students collect and label their own data
- Final result should demonstrate the idea using a **live camera stream**

Expected Project Workflow

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

1. Define the project idea
2. Collect image/video data using a webcam
3. Label the collected data
4. Train or adapt a model
5. Evaluate the result
6. Demonstrate the system with a live camera stream

Project Evaluation

ESIGELEC
SCHOOL OF ENGINEERING
SMART AND CONNECTED SYSTEMS
ROUEN | POITIERS

The project will be evaluated based on:

- **Project completeness**
- Quality of data collection and labeling
- Working implementation
- Live webcam demonstration
- Final presentation on the last lecture day